



Claude Certified Architect Foundations *Cheat Sheets*

These cheat sheets are provided for non-commercial purpose for personal study.

Please do not redistribute or upload these cheat sheets elsewhere.

Good luck on your exam!

Claude Certified Architect *CheatSheet*

Exam

Pro

Agentic Coding Tool (aka Coding Agent) that **reads your codebase, edits files, runs commands,** and integrates with your development tools.

Coding Agents can be available in your terminal, IDE, desktop app, and browser.

Code Harness (Agentic Harness) is a system that wraps an LLM with prompts, tools, and execution environments to turn it into an agent that can take actions and run workflows.

Claude Code is an **agentic coding tool**. It that reads your codebase, edits files, runs commands, and integrates with your development tools.

Agentic Loop

A continuous cycle where an LLM gathers context, takes actions using tools, and verifies results until the goal is achieved.

- Core loop: **Gather Context → Take Action → Verify Results → repeat**
- Uses **tool calls** (functions/APIs) to interact with code, systems, and external services
- **Self-reflects** after each step to decide next action
- Stops only when the **goal is met**
- You can **interrupt or guide** the loop at any time
- Tools enable real actions: read/edit files, run commands, call APIs, test code

Agentic Loop – Models

You can **choose the model** used throughout the agentic loop

Model choice impacts **speed, cost, and reasoning ability**

Key models:

- **default** → auto-selected, recommended
- **sonnet** → balanced, best for most coding tasks
- **opus** → strongest reasoning, slower, higher cost
- **haiku** → fastest, cheapest, for simple tasks

Claude Certified Architect *CheatSheet*

Exam

Pro

- **sonnet[1m]** → large context (long sessions)
- **opusplan** → opus for planning, sonnet for execution

Core idea: Pick the model based on **task complexity vs cost/speed tradeoff**

Claude API Key can be used to control cost and usage for Claude Code.

Useful for **production-use-cases and automated systems**.

The token is generated at: <https://platform.claude.com>

Stop Reason is the **reason why** the Claude Agent has **stopped executing its loop**.

Stop reason will be of two different values:

1. Tool User
2. End Turn

The JSON data returned from a request to the agent will have **"stop_reason"**

stop_reason: "tool_use": Is when the agent stops with the intention to call an external function

stop_reason: "end_turn": Is when the agent has decided to return you a result.

When Claude returns a stop_reason: **"tool_use"** to the program.

- The program needs to parse and send that data to a function. The function will return data.

Then you append the result of the data via "type": **"tool_result"**

Decision Making

- **Pre-configured** → hardcoded logic (if/else, state machines)
- You control **what happens next**
- Tool sequences ≈ **state machine (step-by-step flow)**
- **Model-driven** → give **tools + goal**, Claude decides flow
- Chooses **which tool + when** at runtime
- **Hub-and-spoke** → coordinator controls subagents (no direct communication)

Anti-Patterns (Agentic Loop)

- ❌ Don't parse natural language → brittle, unreliable
- ❌ Don't stop based on text content
- ❌ Don't use arbitrary iteration caps as primary stop
- ✅ Always use stop_reason (tool_use vs end_turn)
- ✅ Iteration cap = **safety only**, not control logic
- ✅ Ignore text until end_turn → then return final answer

Hub-and-Spoke Architecture

- **Coordinator = central brain** → all agents route through it
- Subagents **never talk directly** → no shared context
- Coordinator: **decompose → delegate → aggregate**
- Chooses: **single, sequential, or parallel agents** based on complexity
- Final step: **merge results + resolve conflicts → one response**

Narrow Task Decomposition (Coordinator)

- ❌ Too narrow → **misses important dimensions**
- Subagents have **isolated context** → can't detect gaps
- ✅ Expand tasks to cover **all perspectives before delegating**
- Think: **what's missing?** then add subtasks
- ✅ Validate decomposition (review tool or during aggregation)
- Final step: **check gaps before producing answer**

Dynamic Selection

- ❌ Don't run full pipeline every time
- ✅ Select only needed agents
- Simple → single
- Complex → sequential / parallel
- Use agents **only if they add value**

Partitioning Research

- ❌ Same task → **overlap + wasted tokens**
- ✅ Split into **distinct, non-overlapping scopes**
- Each agent owns a **unique slice of the problem**

Refinement Loop

- Not one-shot → **evaluate, identify gaps, re-delegate**
- Iterate until **coverage meets a defined quality bar**
- Use structured checks (**evaluate_coverage**) to enforce completeness

Observability

- Without coordinator → **untraceable flows, inconsistent failures**
- With coordinator → **all messages and errors centralized**
- Enables **debugging, control, and policy enforcement**

Agent Tool

- Spawns subagents with **isolated context and tool scope**
- "Task" = legacy naming, still appears in code
- Supports **parallel or blocking execution patterns**

Task Tool (Context Passing)

- Subagents receive **only explicitly provided context**
- ❌ No implicit memory or context sharing
- ✅ Prompts must include **all required instructions and data**

Agent Definition

- Blueprint for a subagent: **identity, role, and boundaries**
- Includes: **name, description, system prompt, allowed/disallowed tools**
- Defines **what the agent can do before execution begins**

Fork-Based Session Management

- Copy a conversation state → **branch into independent sessions**
- Each branch explores a **different direction in isolation**
- Prevents cross-contamination while enabling **parallel exploration**

Forking Benefits

- Each session has its own **history and resumable state**
- Runs in **parallel → faster results**
- Fully **isolated → no shared context or leakage**

Claude Code Sessions

- A session = **stateful conversation with the agent**
- Starting claude creates a **new session**, /resume continues one
- Sessions exist **locally or remotely**, and can be continued across environments

Anthropic SDK vs Claude Agent SDK

- **Anthropic SDK** → direct API calls, you control everything
- **Agent SDK** → built-in agent loop + tools, higher-level abstraction

Resuming Sessions

- `/resume` or `claude --resume` **restores a previous session**
- Uses a **session_id** to reload full context/history
- Continuation preserves **state, tools, and prior outputs**

Forking Sessions

- Fork = **branch from a past point into a new session**
- Keeps history **up to that point**, then diverges independently
- Enables **parallel exploration without affecting original**

`/context`

- Shows **token usage + remaining context window**
- Breaks down usage (messages, system, tools, skills)
- Includes **auto-compact buffer** to reserve space for summarization

Compact and Clear

- `/compact` → **summarizes conversation to reduce tokens**
- `/clear` → **resets current chat (keeps files/memory)**
- Used to manage **context limits and prevent overflow**

Rename and Rewind

- `/rename` → **label sessions for easier retrieval**
- `/rewind` → **restore to a previous state in history**
- Useful for **iteration control and undoing bad paths**

Claude Code Settings Scope

- Settings defined via settings.json at multiple levels
- **Managed** → org-wide (highest priority)
- **User** → global personal (~/.claude/settings.json)
- **Project** → shared in repo (.claude/settings.json)
- **Local** → personal, not in git (settings.local.json)

Claude Code Settings



Auth

- API keys + AWS auth helpers
- Force login method / org



Sessions

- Auto-delete old sessions
- Memory + plans storage paths



Env

- Global variables for all runs
- (secrets, configs, flags)



Models

- Set default model
- Limit available models
- Map to providers (e.g. Bedrock)



Output

- Style (e.g. explanatory)
- Language

- Optional git instructions



Git

- Commit / PR attribution
- Co-author toggle



Updates

- Startup announcements
- Update channel (latest / stable)



UI

- Show duration
- Customize spinner text
- Tips + progress bar
- Reduce motion



Agents

- Teammate mode (auto | tmux | inline)



Permissions

- allow → auto run
- ask → confirm
- deny → block
- extra directories + default mode



MCP

- Enable/disable servers
- Allowlist / denylist

Plugins (Managed)

- Allow/block marketplaces
- Custom trust message

Lockdown

- Managed-only → no user/project overrides

Hooks

- On/off all hooks
- Limit URLs + env vars
- Run commands after actions

Misc

- OTEL headers
- Status line
- @file autocomplete
- Respect .gitignore

Permissions Rules

- Levels: **allow** | **ask** | **deny**
- Order: **deny → ask → allow (most restrictive wins)**
- Tools: Bash, Read/Write, WebFetch, MCP, Agents

Bash Wildcard Permissions

- Wildcard: * matches any characters. Works anywhere in command
- Spaces matter (ls * only)

Read & Edit Rules

- Follows .gitignore patterns

- // absolute path
- ~/ home directory
- / project root
- none = current directory

WebFetch Rules

- Controlled by domain
- WebFetch(domain:example.com)
- Can allow / ask / deny per domain

MCP Rules

- **mcp__server** = all tools
- **mcp__server__tool** = specific tool
- Supports wildcards

Bare Tool Rules: No () = full access

- Examples: Read, Bash, WebFetch
- Very broad permissions

Tools

- Agent → subagents
- Bash* → run commands
- Edit* → modify files
- Read → read files
- WebFetch* → fetch URLs
- WebSearch* → search web
- Write* → create/overwrite files

Claude Certified Architect *Cheat Sheet*

Exam **Pro**

- Grep / Glob → search & match
- TaskCreate / TaskGet / TaskList, TaskUpdate / TaskStop / TaskOutput

Permission Modes for tool approval:

- **default** Standard behavior: prompts for permission on first use of each tool
- **acceptEdits** Automatically accepts file edit permissions for the session
- **plan** Plan Mode: Claude can analyze but not modify files or execute commands
- **dontAsk** Auto-denies tools unless pre-approved via /permissions or permissions.allow rules
- **bypassPermissions** Skips all permission prompts (requires safe environment, see warning below)

--**dangerously-skip-permission** is a setting which will tell Claude for a session to **stop prompting you to ask for permissions**:

A **sandbox** is a **security mechanism for separating running programs** for your OS.

A sandbox provides a controlled environment for programs to run with:

- storage and memory scratch space
- limited network access
- restricted ability to inspect the host system

Raw Findings Dilemma

- Passing raw text between agents **loses attribution and provenance**
- Downstream agents can't trace **which source supports which claim**
- Results in **low trust, unverifiable outputs, and weak reliability**

Structured Findings (Solution)

- Use structured objects to keep **content + metadata together**
- Include fields like **id, type, confidence, and source details**
- Enables **traceability, validation, and reliable synthesis across agents**

Parallel Tool Calls

- Run subagents **in parallel when tasks are independent** (no shared dependencies)
- Improves **speed and coverage** by exploring multiple paths at once
- Use **sequential flow only when outputs depend on earlier steps**

Procedural (Rigid) Coordination

- Step-by-step prompts make the coordinator **act like a script runner**
- Breaks when inputs are missing, unexpected, or tasks don't fit the flow
- Leads to **wasted calls and poor adaptability**

Goal-Oriented Coordination

- Define a **clear end goal + quality bar**, not fixed steps
- Coordinator decides **which agents to use, in what order, dynamically**
- Enables **flexibility, better reasoning, and efficient execution**

Quality Criteria Driven

- Use explicit criteria as a **definition of done** (coverage, sources, depth)
- Coordinator evaluates outputs against these before finishing
- If unmet → **identify gaps and re-delegate targeted work**

Prompt-based Guidance

- Instructions guide behavior, **not enforced**
- Model can **drift or ignore rules**
- No guarantee of correct sequence

Programmatic Enforcement (Gates)

- Hard rules make steps **impossible to skip**
- Block execution if prerequisites missing
- Ensures **correct order + reliability**

Hooks Pattern

- Hooks run **before/after tool calls**
- Used to implement **gates + state updates**
- Central place for control logic

Handoff Protocol

- Structured package before escalation
- Includes **context, actions, blockers**
- Avoids digging through history

PreToolUse / PostToolUse

- Pre: validate, enforce rules
- Post: store results, update state
- Enables **controlled, observable flows**

Prompt Chaining

- Fixed sequence of agent steps
- Best for predictable, repeatable workflows

Dynamic Adaptive Decomposition

- Decide next step based on findings
- Flexible, not a fixed sequence

Adaptive Investigation Plan

- Findings generate new questions
- Stop when no new signal emerges

Resuming Sessions

- Model has no memory of past state
- Provide a fresh structured summary

System Prompts vs Tools

- System prompts can override tool intent
- Watch for hidden constraints/conflicts

MCP isError

- **Tool failed ≠ loop failed**
- Agent still receives the result and keeps reasoning
- Use error **type** → **action mapping** (retry / fix / escalate)

Retryable Flag

- **Not all errors should retry**
- **retryable**: true → safe to retry
- **retryable**: false → communicate, don't loop

Subagent Failure Recovery

Bad pattern: propagate everything → coordinator gets useless failure

Better pattern:

- retry locally
- try fallback
- escalate only if exhausted

Too Many Tools: More tools ≠ more capability

- Confuses selection → wrong tool chosen
- Increases hesitation → unnecessary clarifications
- Encourages hallucinated tool combos
 - **Rule: prefer small, general-purpose toolsets over many niche ones**

Tools Specialization Misuse

- ⚠️ *If a tool exists, the model assumes it should be used*
- Tool presence = implicit permission
- **Role mismatch → wrong tool usage** (e.g. synthesis agent searching)

Tool Choice

Controls how much freedom the model has

- **AUTO** → flexible, but unpredictable (Use **AUTO** for general agents)
- **ANY** → guarantees tool use
- **TOOL** → fully deterministic (forced path) (use **TOOL** for strict pipelines / enforcement)
- **NONE** → pure reasoning, no tools

MCP Discovery

● All tools flatten into one list

- No grouping by server
- No awareness of origin
- Just one large decision surface

Risk: tool explosion without structure

Mitigation: limit servers or filter tools before exposure

MCP Tools vs Resources

Two very different things:

- Tools → actions (do something)
- Resources → data (read something)

Built-In Tools

🧠 **Mental model: file system + shell primitives**

- **Read** → load file into context
- **Grep** → search inside files
- **Glob** → find files by pattern
- **Edit** → modify files (needs permission)
- **Bash** → execute commands

JSON Schema Format

📦 **Defines structure, not behavior**

- type → data shape
- properties → fields
- required → must exist
- enum → allowed values

Tool Input Schema

🔴 **The model generates inputs to match this**

- Schema = *how Claude calls your tool*
- Must be **clear + constrained**

Key pieces:

- **enum** → reduces ambiguity
- **required** → enforces completeness
- **description** → guides decision-making

Input Schema Tips (Conditional Logic)

- 🧠 You can simulate logic without code
- Make optional fields *conditionally required* via description
- Example: "category_detail" required when "category" = "other"

Input Schema Tips (Validation Layer)

- 🧱 **Schema ≠ full validation**
- Use tools like **Pydantic** / **Instructor**
- Validate:
 - input to model
 - output from model

Prompting with Explicit Criteria

- Vague prompts → **unpredictable results**
- Specify **scope, constraints, success condition**
- 🟡 Tell the model what *not* to change

False Positives (Vague Prompting)

- Vague prompts → over-flagging + wasted time
- Model guesses patterns instead of rules
- 🟡 Define **exact criteria** for detection

Few Shot Prompting

- Examples → **anchor behavior + format**
- Include edge cases (good + bad)
- ● Reduces hallucination + improves consistency

Large Context Problems

Progressive Summarization

- Info degrades across steps (numbers lost)
- Extract + re-inject key data

Lost in the Middle

- Model ignores middle of long context
- Chunk + process in sections

Tool Result Bloat

- Too much data → noise + token waste
- Filter before passing

Conversation History Growth

- Context grows uncontrollably
- Summarize or trim history

Scenario Examples

Customer Support Agent

- Uses tools (MCP)
- Resolves or escalates

Claude Code (Dev)

- Generates / fixes code
- Plan vs execute

Multi-Agent System

- Coordinator + subagents
- Split tasks

Dev Productivity Agent

- Uses built-in tools
- Automates work

CI/CD

- Reviews + tests
- Avoid false positives